

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 4**



DOUBLE LINK LIST

Oleh:

Muhammad Rafly Ash-Shadiq

NIM. 2510817310007

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 4

Laporan Praktikum Algoritma & Struktur Data Modul 4: Double Link List ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rafly Ash-Shadiq
NIM : 2510817310007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom., M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
SOAL 1	1
A. Pembahasan.....	8
SOAL 2	21
A. Pembahasan.....	21
SOAL 3	23
A. Pembahasan.....	23
SOAL 4	25
A. Pembahasan.....	25
LINK GITHUB	37

DAFTAR GAMBAR

Gambar 1 Screenshot Output Soal 1	18
Gambar 2 Screenshot Output Soal 4	35
Gambar 3 Screenshot Output Soal 4	35

DAFTAR TABEL

Tabel 1 Source Code Jawaban Soal 1	8
Tabel 2 Source Code Jawaban Soal 4	25

SOAL 1

Lengkapi coding pada function tambahDepanH() agar bisa berjalan dengan lancar. running, simpan program, dan screenshoot hasil running !

```
1  #include <conio.h>
2  #include <iostream>
3  #include <stdlib.h>
4
5  using namespace std;
6
7  typedef struct TNode {
8      string data;
9      TNode *next;
10     TNode *prev;
11 };
12
13 TNode *head, *tail;
14
15 int pil, menu;
16 char pilihan[1];
17 string dataBaru;
18
19 void inith();
20 void inithT();
21 int isEmptyH();
22 int isEmptyHT();
23
24 void tambahDepanH();
25 void tambahDepanHT();
26 void tambahBelakangH();
27 void tambahBelakangHT();
28 void hapusDepanH();
29 void hapusDepanHT();
30 void hapusBelakangH();
31 void hapusBelakangHT();
32 void tampilkanH();
33 void tampilkanHT();
34 void clearH();
35 void clearHT();
36
37 int main()
38 {
39     menu:
40     cout<<"Double Linked List Non Circular (DLLNC)"<<endl;
41     cout<<"===== "<<endl;
42     cout<<"Silahkan pilih program DLLNC yang ingin dijalankan!"<<endl;
43     cout<<"1. DLLNC dengan Head"<<endl;
44     cout<<"2. DLLNC dengan Head dan Tail"<<endl;
45     cout<<"3. Quit"<<endl;
46     cout<<"Pilihan : ";
47     cin>>menu;
48     system("cls");
```

```

49  if(menu==1){
50      do {
51          cout<<"Double Linked List Non Circular (DLLNC) (Head)"<<endl;
52          cout<<"===== "<<endl;
53          cout<<"1. Tambah Depan"<<endl;
54          cout<<"2. Tambah Belakang"<<endl;
55          cout<<"3. Tampilkan Data"<<endl;
56          cout<<"4. Hapus Depan"<<endl;
57          cout<<"5. Hapus Belakang"<<endl;
58          cout<<"6. Reset"<<endl;
59          cout<<"7. Kembali ke Menu"<<endl;
60          cout<<"Pilihan : ";
61          cin>>pilihan;
62          pil=atoi(pilihan);

```

```

63
64      switch(pil) {
65          case 1:
66              tambahDepanH();
67              break;
68          case 2:
69              tambahBelakangH();
70              break;
71          case 3:
72              tampilkanH();
73              break;
74          case 4:
75              hapusDepanH();
76              break;
77          case 5:
78              hapusBelakangH();
79              break;
80          case 6:
81              clearH();
82              break;
83          default:
84              system("cls");
85              goto menu;
86      }
87
88      cout<<"\npress any key to continue"<<endl;
89      getch();
90      system("cls");
91
92      } while (pil<7);
93  } else if(menu==2){
94      do {

```

```

95     cout<<"Double Linked List Non Circular (DLLNC) (Head dan Tail)"<<endl;
96     cout<<"-----"<<endl;
97     cout<<"1. Tambah Depan"<<endl;
98     cout<<"2. Tambah Belakang"<<endl;
99     cout<<"3. Tampilkan Data"<<endl;
100    cout<<"4. Hapus Depan"<<endl;
101    cout<<"5. Hapus Belakang"<<endl;
102    cout<<"6. Reset"<<endl;
103    cout<<"7. Kembali ke Menu"<<endl;
104    cout<<"Pilihan : ";
105    cin>>pilihan;
106    pil=atoi(pilihan);
107
108    switch(pil) {
109    case 1:
110        tambahDepanHT();
111        break;
112    case 2:
113        tambahBelakangHT();
114        break;
115    case 3:
116        tampilkanHT();
117        break;
118    case 4:
119        hapusDepanHT();
120        break;
121    case 5:
122        hapusBelakangHT();
123        break;
124    case 6:
125        clearHT();
126        break;
127    default:
128        system("cls");
129        goto menu;
130    }
131
132    cout<<"\npress any key to continue"<<endl;
133    getch();
134    system("cls");
135
136    } while (pil<7);
137    } else {
138        cout<<"\nTERIMA KASIH"<<endl;
139        cout<<"Program was made by Nama (NIM)."<<endl;
140    }
141    }
142

```



```

143 void initH(){
144     head = NULL;
145 }
146
147 void initHT(){
148     head = NULL;
149     tail = NULL;
150 }
151
152 int isEmptyH(){
153     if(head == NULL) return 1;
154     else return 0;
155 }
156
157 int isEmptyHT(){
158     if(tail == NULL) return 1;
159     else return 0;
160 }
161
162 void tambahDepanH() {
163     cout<<"Masukkan data : ";
164
165
166
167
168
169
170
171
172
173
174
175
176
177     cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan.";
178 }
179
180 void tambahDepanHT() {
181     cout<<"Masukkan data : ";
182     cin>>dataBaru;
183     TNode *baru;
184     baru = new TNode;
185     baru->data = dataBaru;
186     baru->next = NULL;
187     baru->prev = NULL;
188     if(isEmptyHT() == 1) {
189         head = baru;
190         tail = baru;

```

```

191     } else {
192         baru->next = head;
193         head->prev = baru;
194         head = baru;
195     }
196     cout << "Data \\"<<dataBaru<<\" berhasil dimasukkan di bagian depan.";
197 }
198
199 void tambahBelakangH() {
200     cout<<"Masukkan data : ";
201     cin>>dataBaru;
202     TNode *baru, *bantu;
203     baru = new TNode;
204     baru->data = dataBaru;
205     baru->next = NULL;
206     baru->prev = NULL;
207     if(isEmptyH() == 1) {
208         head = baru;
209     } else {
210         bantu = head;
211         while(bantu->next != NULL){
212             bantu = bantu->next;
213         }
214         bantu->next = baru;
215         baru->prev = bantu;
216     }
217     cout << "Data \\"<<dataBaru<<\" berhasil dimasukkan di bagian belakang.";
218 }
219

```

```

220 void tambahBelakangHT() {
221     cout<<"Masukkan data : ";
222     cin>>dataBaru;
223     TNode *baru;
224     baru = new TNode;
225     baru->data = dataBaru;
226     baru->next = NULL;
227     baru->prev = NULL;
228     if(isEmptyHT() == 1) {
229         head = baru;
230         tail = baru;
231     } else {
232         tail->next = baru;
233         baru->prev = tail;
234         tail = baru;
235     }
236     cout << "Data \\"<<dataBaru<<\" berhasil dimasukkan di bagian belakang.";
237 }

```

```

238
239 void tampilkanH() {
240     TNode *bantu;
241     bantu = head;
242     if(isEmptyH() == 0) {
243         while(bantu != NULL) {
244             cout<<bantu->data<<' ';
245             bantu = bantu->next;
246         }
247         cout<<endl;
248     } else cout<<"Tidak terdapat data pada Linked List";
249 }
250
251 void tampilkanHT() {
252     TNode *bantu;
253     bantu = head;
254     if(isEmptyHT() == 0) {
255         while(bantu != tail->next) {
256             cout<<bantu->data<<' ';
257             bantu = bantu->next;
258         }
259         cout<<endl;
260     } else cout<<"Tidak terdapat data pada Linked List";
261 }
262
263 void hapusDepanH() {
264     TNode *hapus;
265     string data;
266     if(isEmptyH() == 0) {
267         hapus = head;
268         data = hapus->data;
269         if(head->next != NULL) {
270             head = head->next;
271             head->prev = NULL;
272         } else {
273             inith();
274         }
275         delete hapus;
276         cout<<"Data \""<<data<<"\" yang berada di depan telah berhasil dihapus.";
277     } else cout<<"Tidak terdapat data pada Linked List";
278 }
279
280 void hapusDepanHT() {
281     TNode *hapus;
282     string data;
283     if(isEmptyHT() == 0) {
284         hapus = head;
285         data = hapus->data;

```

```

286         if(head->next != NULL) {
287             head = head->next;
288             head->prev = NULL;
289         } else {
290             initHT();
291         }
292         delete hapus;
293         cout<<"Data \\"<<data<<" yang berada di depan telah berhasil dihapus.";
294     } else cout<<"Tidak terdapat data pada Linked List";
295 }
296
297 void hapusBelakangH() {
298     TNode *hapus;
299     string data;
300     if(isEmptyH() == 0) {
301         hapus = head;
302         while(hapus->next != NULL){
303             hapus = hapus->next;
304         }
305         data = hapus->data;
306         if(head->next != NULL) {
307             hapus->prev->next = NULL;
308         } else {
309             initH();
310         }
311         delete hapus;
312         cout<<"Data \\"<<data<<" yang berada di belakang telah berhasil dihapus.";
313     } else cout<<"Tidak terdapat data pada Linked List";
314 }
315
316 void hapusBelakangHT() {
317     TNode *hapus;
318     string data;
319     if(isEmptyHT() == 0) {
320         hapus = tail;
321         data = hapus->data;
322         if(head->next != NULL) {
323             tail = tail->prev;
324             tail->next = NULL;
325         } else {
326             initHT();
327         }
328         delete hapus;
329         cout<<"Data \\"<<data<<" yang berada di belakang telah berhasil dihapus.";
330     } else cout<<"Tidak terdapat data pada Linked List";
331 }
332
333 void clearH() {

```

```

334     TNode *bantu, *hapus;
335     bantu = head;
336     while(bantu != NULL) {
337         hapus = bantu;
338         bantu = bantu->next;
339         delete hapus;
340     }
341     inith();
342     cout<<"Seluruh data pada Linked List telah dibersihkan.";
343 }
344
345 void clearHT() {
346     TNode *bantu, *hapus;
347     bantu = head;
348     while(bantu != NULL) {
349         hapus = bantu;
350         bantu = bantu->next;
351         delete hapus;
352     }
353     inithHT();
354     cout<<"Seluruh data pada Linked List telah dibersihkan.";
355 }

```

A. Pembahasan

Tabel 1 Source Code Jawaban Soal 1

1.	#include <conio.h>
2.	#include <iostream>
3.	#include <stdlib.h>
4.	
5.	using namespace std;
6.	
7.	struct TNode {
8.	string data;
9.	TNode *next;
10.	TNode *prev;
11.	};
12.	
13.	TNode *head, *tail;
14.	
15.	int pil, menu;
16.	char pilihan[1];
17.	string dataBaru;
18.	
19.	void inith();
20.	void inithT();

```

21. int isEmptyH();
22. int isEmptyHT();
23.
24. void tambahDepanH();
25. void tambahDepanHT();
26. void tambahBelakangH();
27. void tambahBelakangHT();
28. void hapusDepanH();
29. void hapusDepanHT();
30. void hapusBelakangH();
31. void hapusBelakangHT();
32. void tampilkanH();
33. void tampilkanHT();
34. void clearH();
35. void clearHT();
36.
37. int main()
38. {
39.     menu:
40.     cout<<"Double    Linked    List    Non    Circular
(DLLNC) "<<endl;
41.
42.     cout<<"===== "<<en
dl;
43.     cout<<"Silahkan pilih program DLLNC yang ingin
dijalankan!"<<endl;
44.     cout<<"1. DLLNC dengan Head"<<endl;
45.     cout<<"2. DLLNC dengan Head dan Tail"<<endl;
46.     cout<<"3. Quit"<<endl;
47.     cout<<"Pilihan : ";
48.     cin>>menu;
49.     system("cls");
50.     if(menu==1){
51.         do {
52.             cout<<"Double Linked List Non Circular
(DLLNC) (Head) "<<endl;
53.
54.             cout<<"=====
=="<<endl;
55.             cout<<"1. Tambah Depan"<<endl;
56.             cout<<"2. Tambah Belakang"<<endl;

```

```

55.         cout<<"3. Tampilkan Data"<<endl;
56.         cout<<"4. Hapus Depan"<<endl;
57.         cout<<"5. Hapus Belakang"<<endl;
58.         cout<<"6. Reset"<<endl;
59.         cout<<"7. Kembali ke Menu"<<endl;
60.         cout<<"Pilihan : ";
61.         cin>>pilihan;
62.         pil=atoi(pilihan);
63.
64.         switch(pil) {
65.             case 1:
66.                 tambahDepanH();
67.                 break;
68.             case 2:
69.                 tambahBelakangH();
70.                 break;
71.             case 3:
72.                 tampilkanH();
73.                 break;
74.             case 4:
75.                 hapusDepanH();
76.                 break;
77.             case 5:
78.                 hapusBelakangH();
79.                 break;
80.             case 6:
81.                 clearH();
82.                 break;
83.             default:
84.                 system("cls");
85.                 goto menu;
86.         }
87.
88.         cout<<"\npress any key to
continue"<<endl;
89.         getch();
90.         system("cls");
91.
92.     } while (pil<7);
93. } else if(menu==2){

```

```

94.         do {
95.             cout<<"Double Linked List Non Circular
(DLLNC) (Head dan Tail)"<<endl;
96.
cout<<"=====
===== "<<endl;
97.             cout<<"1. Tambah Depan"<<endl;
98.             cout<<"2. Tambah Belakang"<<endl;
99.             cout<<"3. Tampilkan Data"<<endl;
100.            cout<<"4. Hapus Depan"<<endl;
101.            cout<<"5. Hapus Belakang"<<endl;
102.            cout<<"6. Reset"<<endl;
103.            cout<<"7. Kembali ke Menu"<<endl;
104.            cout<<"Pilihan : ";
105.            cin>>pilihan;
106.            pil=atoi(pilihan);
107.
108.            switch(pil) {
109.                case 1:
110.                    tambahDepanHT();
111.                    break;
112.                case 2:
113.                    tambahBelakangHT();
114.                    break;
115.                case 3:
116.                    tampilkanHT();
117.                    break;
118.                case 4:
119.                    hapusDepanHT();
120.                    break;
121.                case 5:
122.                    hapusBelakangHT();
123.                    break;
124.                case 6:
125.                    clearHT();
126.                    break;
127.                default:
128.                    system("cls");
129.                    goto menu;
130.            }

```



```

131.
132.         cout<<"\npress any key to
continue"<<endl;
133.         getch();
134.         system("cls");
135.
136.     } while (pil<7);
137. } else {
138.     cout<<"\nTERIMA KASIH"<<endl;
139.     cout<<" Muhammad Rafly Ash-Shadiq
(2510817310007) ."<<endl;
140. }
141. }
142.
143. void initH(){
144.     head = NULL;
145. }
146.
147. void initHT(){
148.     head = NULL;
149.     tail = NULL;
150. }
151.
152. int isEmptyH(){
153.     if(head == NULL) return 1;
154.     else return 0;
155. }
156.
157. int isEmptyHT(){
158.     if(tail == NULL) return 1;
159.     else return 0;
160. }
161.
162. void tambahDepanH() {
163.     cout<<"Masukkan data : ";
164.     cin>>dataBaru;
165.     TNode *baru;
166.     baru = new TNode;
167.     baru->data = dataBaru;
168.     baru->next = NULL;

```

```

169.     baru->prev = NULL;
170.     if(isEmptyH() == 1) {
171.         head = baru;
172.     } else {
173.         baru->next = head;
174.         head->prev = baru;
175.         head = baru;
176.     }
177.     cout << "Data \""<<dataBaru<<"\" berhasil
dimasukkan di bagian depan.";
178. }
179.
180. void tambahDepanHT() {
181.     cout<<"Masukkan data : ";
182.     cin>>dataBaru;
183.     TNode *baru;
184.     baru = new TNode;
185.     baru->data = dataBaru;
186.     baru->next = NULL;
187.     baru->prev = NULL;
188.     if(isEmptyHT() == 1) {
189.         head = baru;
190.         tail = baru;
191.     } else {
192.         baru->next = head;
193.         head->prev = baru;
194.         head = baru;
195.     }
196.     cout << "Data \""<<dataBaru<<"\" berhasil
dimasukkan di bagian depan.";
197. }
198.
199. void tambahBelakangH() {
200.     cout<<"Masukkan data : ";
201.     cin>>dataBaru;
202.     TNode *baru, *bantu;
203.     baru = new TNode;
204.     baru->data = dataBaru;
205.     baru->next = NULL;
206.     baru->prev = NULL;

```

```

207.     if(isEmptyH() == 1) {
208.         head = baru;
209.     } else {
210.         bantu = head;
211.         while(bantu->next != NULL){
212.             bantu = bantu->next;
213.         }
214.         bantu->next = baru;
215.         baru->prev = bantu;
216.     }
217.     cout << "Data \""<<dataBaru<<"\" berhasil
dimasukkan di bagian belakang.";
218. }
219.
220. void tambahBelakangHT() {
221.     cout<<"Masukkan data : ";
222.     cin>>dataBaru;
223.     TNode *baru;
224.     baru = new TNode;
225.     baru->data = dataBaru;
226.     baru->next = NULL;
227.     baru->prev = NULL;
228.     if(isEmptyHT() == 1) {
229.         head = baru;
230.         tail = baru;
231.     } else {
232.         tail->next = baru;
233.         baru->prev = tail;
234.         tail = baru;
235.     }
236.     cout << "Data \""<<dataBaru<<"\" berhasil
dimasukkan di bagian belakang.";
237. }
238.
239. void tampilkanH() {
240.     TNode *bantu;
241.     bantu = head;
242.     if(isEmptyH() == 0) {
243.         while(bantu != NULL) {
244.             cout<<bantu->data<<' ';

```

```

245.         bantu = bantu->next;
246.     }
247.     cout<<endl;
248.     } else cout<<"Tidak terdapat data pada Linked
List";
249. }
250.
251. void tampilkanHT() {
252.     TNode *bantu;
253.     bantu = head;
254.     if(isEmptyHT() == 0) {
255.         while(bantu != tail->next) {
256.             cout<<bantu->data<<' ';
257.             bantu = bantu->next;
258.         }
259.         cout<<endl;
260.     } else cout<<"Tidak terdapat data pada Linked
List";
261. }
262.
263. void hapusDepanH() {
264.     TNode *hapus;
265.     string data;
266.     if(isEmptyH() == 0) {
267.         hapus = head;
268.         data = hapus->data;
269.         if(head->next != NULL) {
270.             head = head->next;
271.             head->prev = NULL;
272.         } else {
273.             initH();
274.         }
275.         delete hapus;
276.         cout<<"Data \""<<data<<"\" yang berada di
depan telah berhasil dihapus.";
277.     } else cout<<"Tidak terdapat data pada Linked
List";
278. }
279.
280. void hapusDepanHT() {

```

```

281.     TNode *hapus;
282.     string data;
283.     if(isEmptyHT() == 0) {
284.         hapus = head;
285.         data = hapus->data;
286.         if(head->next != NULL) {
287.             head = head->next;
288.             head->prev = NULL;
289.         } else {
290.             initHT();
291.         }
292.         delete hapus;
293.         cout<<"Data \""<<data<<"\" yang berada di
depan telah berhasil dihapus.";
294.     } else cout<<"Tidak terdapat data pada Linked
List";
295. }
296.
297. void hapusBelakangH() {
298.     TNode *hapus;
299.     string data;
300.     if(isEmptyH() == 0) {
301.         hapus = head;
302.         while(hapus->next != NULL){
303.             hapus = hapus->next;
304.         }
305.         data = hapus->data;
306.         if(head->next != NULL) {
307.             hapus->prev->next = NULL;
308.         } else {
309.             initH();
310.         }
311.         delete hapus;
312.         cout<<"Data \""<<data<<"\" yang berada di
belakang telah berhasil dihapus.";
313.     } else cout<<"Tidak terdapat data pada Linked
List";
314. }
315.
316. void hapusBelakangHT() {

```

```

317.     TNode *hapus;
318.     string data;
319.     if(isEmptyHT() == 0) {
320.         hapus = tail;
321.         data = hapus->data;
322.         if(head->next != NULL) {
323.             tail = tail->prev;
324.             tail->next = NULL;
325.         } else {
326.             initHT();
327.         }
328.         delete hapus;
329.         cout<<"Data \""<<data<<"\" yang berada di
belakang telah berhasil dihapus.";
330.     } else cout<<"Tidak terdapat data pada Linked
List";
331. }
332.
333. void clearH() {
334.     TNode *bantu, *hapus;
335.     bantu = head;
336.     while(bantu != NULL) {
337.         hapus = bantu;
338.         bantu = bantu->next;
339.         delete hapus;
340.     }
341.     initH();
342.     cout<<"Seluruh data pada Linked List telah
dibersihkan.";
343. }
344.
345. void clearHT() {
346.     TNode *bantu, *hapus;
347.     bantu = head;
348.     while(bantu != NULL) {
349.         hapus = bantu;
350.         bantu = bantu->next;
351.         delete hapus;
352.     }
353.     initHT();

```

```

354.         cout<<"Seluruh data pada Linked List telah
           dibersihkan.";
355.     }

```

```

PS C:\Users\Lenovo LOQ\OneDrive\Documents\PRAKTIKUM ALGO 1> cd "C:\Users\Lenovo LOQ\OneDrive\Documents\PRAKTIKUM ALGO 1\Modul_4_Algo\" ; if ($?) { g++ PRAK401_2510817310007_MuhammadRaflyAsh-Shadiq.C++ -o PRAK401_2510817310007_MuhammadRaflyAsh-Shadiq } ; if ($?) { .\PRAK401_2510817310007_MuhammadRaflyAsh-Shadiq }
Double Linked List Non Circular (DLNC)
=====
Silahkan pilih program DLNC yang ingin dijalankan!
1. DLNC dengan Head
2. DLNC dengan Head dan Tail
3. Quit
Pilihan :

```

Gambar 1 Screenshot Output Soal 1

Pada baris 1 `#include <conio.h>` digunakan untuk memanggil fungsi seperti `getch()` yang berfungsi menahan layar sebelum program berlanjut. Pada baris 2 `#include <iostream>` digunakan untuk proses input dan output seperti `cin` dan `cout`. Pada baris 3 `#include <stdlib.h>` digunakan untuk fungsi `atoi()` yang berfungsi mengubah data bertipe karakter menjadi bilangan bulat. Pada baris 5 `using namespace std;` digunakan agar tidak perlu menuliskan `std::` secara berulang. Pada baris 7–11 dideklarasikan `struct TNode` yang merupakan bentuk node dalam Double Linked List. Di dalamnya terdapat data sebagai tempat penyimpanan nilai, `next` sebagai penunjuk ke node berikutnya, dan `prev` sebagai penunjuk ke node sebelumnya. Keterhubungan `next` dan `prev` inilah yang membuat struktur ini bisa berjalan dua arah. Pada baris 13 dideklarasikan pointer `head` dan `tail` yang masing-masing berfungsi sebagai penunjuk node pertama dan node terakhir. Pada baris 15–17 dideklarasikan variabel `pil`, `menu`, `pilihan`, dan `dataBaru` yang digunakan untuk mengatur jalannya program serta menyimpan input pengguna. Pada baris 19–35 merupakan deklarasi seluruh function yang akan digunakan. Hal ini bertujuan agar fungsi dapat dikenali sebelum dipanggil dalam `main()`. Pada baris 37–141 merupakan fungsi utama `main()`. Pada baris 39 terdapat label `menu:` yang digunakan untuk kembali ke menu utama. Baris 40–47 menampilkan pilihan jenis program, kemudian input dimasukkan ke variabel `menu`. Baris 48 membersihkan layar. Jika `menu == 1`, maka program masuk ke blok pada baris 49–92 yang menggunakan Double Linked List dengan head saja. Di dalamnya terdapat

perulangan do-while yang menampilkan menu operasi. Input dimasukkan ke pilihan (baris 61), lalu dikonversi menjadi integer menggunakan `atoi()` (baris 62), kemudian diproses menggunakan switch-case (baris 64–86). Setiap pilihan akan memanggil function tertentu. Jika `menu == 2`, maka program menjalankan versi dengan head dan tail pada baris 93–136. Struktur alurnya sama, hanya berbeda pada function yang digunakan. Pada baris 143–150 terdapat fungsi inisialisasi `initH()` dan `initHT()` untuk mengosongkan linked list. Pada baris 152–160 terdapat fungsi `isEmptyH()` dan `isEmptyHT()` untuk mengecek apakah list kosong. Baris 162–355 berisi seluruh function utama seperti tambah, hapus, tampil, dan clear yang akan dijelaskan lebih lanjut pada bagian berikutnya.

Function `initH()` dan `initHT()` merupakan langkah awal dalam pengelolaan data. `initH()` hanya mengatur `head = NULL`, sedangkan `initHT()` mengatur head dan tail menjadi `NULL`. Keterhubungan sintaks di sini sederhana, yaitu memastikan bahwa tidak ada node yang terhubung sebelum operasi dimulai. Function `isEmptyH()` dan `isEmptyHT()` berfungsi untuk mengecek kondisi linked list. Sintaks `if(head == NULL)` atau `if(tail == NULL)` digunakan sebagai dasar keputusan. Nilai yang dikembalikan akan digunakan oleh function lain sebagai syarat sebelum melakukan operasi, sehingga mencegah kesalahan seperti menghapus data saat list kosong. Function `tambahDepanH()` dimulai dengan input data, kemudian membuat node baru menggunakan `new TNode`. Setelah itu, nilai dimasukkan ke dalam data, dan pointer `next` serta `prev` diatur ke `NULL`. Jika list kosong, maka head langsung menunjuk ke node baru. Jika tidak kosong, maka terjadi keterhubungan sintaks di mana `baru->next = head` menghubungkan node baru ke node lama, lalu `head->prev = baru` menghubungkan balik dari node lama ke node baru. Terakhir, `head = baru` memperbarui posisi awal. Di sini terlihat bahwa setiap sintaks saling berhubungan untuk menjaga struktur dua arah. Function `tambahDepanHT()` memiliki alur yang sama, tetapi jika kosong maka head dan tail langsung diisi node baru. Hal ini menunjukkan keterhubungan antara dua pointer utama yang selalu diperbarui bersama saat kondisi awal. Function `tambahBelakangH()` dimulai dengan pembuatan node baru,

lalu jika list tidak kosong, program menggunakan pointer bantu untuk menelusuri node dari head hingga akhir menggunakan perulangan while. Setelah node terakhir ditemukan, sintaks `bantu->next = baru` menghubungkan node lama ke node baru, dan `baru->prev = bantu` menghubungkan balik. Di sini terlihat relasi antara traversal (while) dan penghubungan node. Function `tambahBelakangHT()` lebih sederhana karena langsung menggunakan tail. Sintaks `tail->next = baru` dan `baru->prev = tail` menunjukkan hubungan dua arah, lalu `tail = baru` memperbarui posisi akhir. Ini menunjukkan bahwa penggunaan tail menghilangkan kebutuhan traversal. Function `tampilkanH()` dan `tampilkanHT()` menggunakan pointer bantu untuk menelusuri data. Sintaks `while(bantu != NULL)` menunjukkan proses iterasi dari awal hingga akhir. Setiap langkah menggunakan `bantu = bantu->next` untuk berpindah node. Hubungan antara sintaks ini menunjukkan bagaimana data dibaca secara berurutan. Function `hapusDepanH()` dimulai dengan menyimpan node head ke variabel sementara. Jika terdapat lebih dari satu node, maka `head = head->next` memindahkan posisi awal, lalu `head->prev = NULL` memutus hubungan ke node lama. Jika hanya satu node, maka dipanggil `initH()`. Sintaks `delete hapus` digunakan untuk menghapus node dari memori. Di sini terlihat hubungan antara pemindahan pointer dan penghapusan data. Function `hapusDepanHT()` memiliki alur yang sama, tetapi juga memperhatikan tail jika hanya terdapat satu node. Function `hapusBelakangH()` menggunakan pointer bantu untuk menelusuri hingga node terakhir. Setelah ditemukan, sintaks `hapus->prev->next = NULL` memutus hubungan dari node sebelumnya. Ini menunjukkan keterhubungan antara pointer prev dan next. Function `hapusBelakangHT()` lebih sederhana karena langsung menggunakan tail. Sintaks `tail = tail->prev` memindahkan posisi akhir, lalu `tail->next = NULL` memutus hubungan node lama. Function `clearH()` dan `clearHT()` menggunakan perulangan untuk menghapus seluruh node satu per satu. Sintaks `bantu = bantu->next` digunakan untuk berpindah, sedangkan `delete hapus` menghapus node. Setelah semua selesai, dipanggil fungsi inisialisasi untuk mengosongkan list

SOAL 2

Apa fungsi next pada coding?

A. Pembahasan

Pada kode Double Linked List Non Circular (DLLNC) yang diberikan, pointer next memiliki fungsi utama sebagai penghubung antar node ke arah depan (forward link). Artinya, setiap node menggunakan next untuk menunjuk ke node berikutnya dalam urutan linked list. Secara konsep, linked list adalah struktur data yang tidak disimpan secara berurutan di memori, sehingga hubungan antar data tidak bergantung pada posisi indeks seperti array, tetapi bergantung pada pointer. Di sinilah peran next menjadi sangat penting, karena tanpa next, node tidak akan tahu node mana yang berada setelahnya. Jika dilihat pada struktur struct TNode yang berisi data, next, dan prev, pointer next berfungsi sebagai jalur maju (forward traversal), sedangkan prev sebagai jalur mundur, sehingga keduanya saling melengkapi dalam double linked list.

Dalam implementasi program, saat menambah data di depan, next digunakan untuk menghubungkan node baru ke node lama seperti pada sintaks baru->next = head, sehingga node baru tetap terhubung dengan list yang sudah ada. Saat menambah data di belakang, next digunakan untuk menyambungkan node terakhir ke node baru seperti tail->next = baru. Pada proses menampilkan data, perulangan berjalan menggunakan next melalui perintah seperti bantu = bantu->next, yang berarti traversal bergerak dari node pertama ke node berikutnya hingga mencapai akhir. Pada saat menghapus node, next juga berperan dalam mengatur ulang hubungan antar node agar struktur linked list tetap tersambung setelah ada node yang dihapus.

Secara keseluruhan, pointer next memastikan bahwa setiap node tetap terhubung dalam satu rangkaian, memungkinkan proses penelusuran data dari awal hingga akhir, serta mendukung seluruh operasi seperti penambahan, penghapusan, dan penampilan data agar berjalan dengan benar. Tanpa adanya next atau jika

pengaturannya salah, maka hubungan antar node akan terputus sehingga linked list tidak dapat berfungsi sebagaimana mestinya. Dengan demikian, dapat disimpulkan bahwa fungsi next adalah sebagai pointer penghubung ke node berikutnya yang menjadi dasar terbentuknya struktur linked list dan jalannya seluruh algoritma di dalam program.

SOAL 3

Apa fungsi prev pada coding?

A. Pembahasan

Pada kode Double Linked List Non Circular (DLLNC) yang diberikan, pointer prev memiliki fungsi utama sebagai penghubung antar node ke arah belakang (backward link). Artinya, setiap node menggunakan prev untuk menunjuk ke node sebelumnya dalam urutan linked list. Berbeda dengan single linked list yang hanya memiliki satu arah hubungan, pada double linked list keberadaan prev memungkinkan setiap node mengetahui node sebelum dirinya, sehingga struktur data menjadi dua arah (bidirectional). Hal ini membuat proses traversal tidak hanya bisa dilakukan dari depan ke belakang menggunakan next, tetapi juga bisa dari belakang ke depan menggunakan prev.

Jika dilihat pada struktur struct TNode yang berisi data, next, dan prev, pointer prev berfungsi sebagai jalur mundur yang melengkapi fungsi next. Dalam implementasi program, saat menambah data di depan, prev digunakan untuk mengatur hubungan node lama agar menunjuk kembali ke node baru, seperti pada sintaks `head->prev = baru`, sehingga hubungan dua arah tetap terjaga. Saat menambah data di belakang, prev digunakan untuk menghubungkan node baru dengan node sebelumnya, misalnya `baru->prev = tail`, sehingga node baru mengetahui posisi sebelumnya dalam list.

Pada proses penghapusan, prev juga berperan penting dalam menjaga keterhubungan antar node. Misalnya saat menghapus node di belakang, pointer prev digunakan untuk mengakses node sebelum tail, kemudian memperbarui hubungan dengan memutus node terakhir dan menjadikan node sebelumnya sebagai tail baru. Selain itu, dalam proses traversal tertentu, prev memungkinkan pergerakan mundur

tanpa harus mengulang dari head, yang membuat beberapa operasi menjadi lebih efisien.

Secara keseluruhan, pointer prev memastikan bahwa setiap node memiliki hubungan ke node sebelumnya, sehingga struktur linked list menjadi fleksibel dan dapat ditelusuri dari dua arah. Tanpa adanya prev, maka linked list hanya bisa berjalan satu arah seperti single linked list, dan beberapa operasi seperti penghapusan dari belakang akan menjadi lebih sulit atau kurang efisien. Dengan demikian, dapat disimpulkan bahwa fungsi prev adalah sebagai pointer penghubung ke node sebelumnya yang mendukung traversal dua arah serta menjaga keterhubungan data dalam struktur double linked list.

SOAL 4

Gantilah baris 244 dan 256 dari `cout<<"bantu->data"<<" ";` menjadi `cout<<"head->data"<<" ";` lalu jawab pertanyaan berikut :

- A. Apa yang terjadi jika anda menambahkan beberapa data pada program lalu tampilkan datanya, dan screenshoot hasilnya.
- B. Jelaskan mengapa hal tersebut bisa terjadi dan data apa yang ditampilkan oleh program?

A. Pembahasan

Tabel 2 Source Code Jawaban Soal 4

1.	<code>#include <conio.h></code>
2.	<code>#include <iostream></code>
3.	<code>#include <stdlib.h></code>
4.	
5.	<code>using namespace std;</code>
6.	
7.	<code>struct TNode {</code>
8.	<code> string data;</code>
9.	<code> TNode *next;</code>
10.	<code> TNode *prev;</code>
11.	<code>};</code>
12.	
13.	<code>TNode *head, *tail;</code>
14.	
15.	<code>int pil, menu;</code>
16.	<code>char pilihan[1];</code>
17.	<code>string dataBaru;</code>
18.	
19.	<code>void initH();</code>
20.	<code>void initHT();</code>
21.	<code>int isEmptyH();</code>
22.	<code>int isEmptyHT();</code>
23.	
24.	<code>void tambahDepanH();</code>
25.	<code>void tambahDepanHT();</code>

```

26. void tambahBelakangH();
27. void tambahBelakangHT();
28. void hapusDepanH();
29. void hapusDepanHT();
30. void hapusBelakangH();
31. void hapusBelakangHT();
32. void tampilkanH();
33. void tampilkanHT();
34. void clearH();
35. void clearHT();
36.
37. int main()
38. {
39.     menu:
40.         cout<<"Double Linked List Non Circular
(DLLNC) "<<endl;
41.         cout<<"===== "<<
endl;
42.         cout<<"Silahkan pilih program DLLNC yang ingin
dijalankan!"<<endl;
43.         cout<<"1. DLLNC dengan Head"<<endl;
44.         cout<<"2. DLLNC dengan Head dan Tail"<<endl;
45.         cout<<"3. Quit"<<endl;
46.         cout<<"Pilihan : ";
47.         cin>>menu;
48.         system("cls");
49.         if(menu==1){
50.             do {
51.                 cout<<"Double Linked List Non Circular
(DLLNC) (Head) "<<endl;
52.                 cout<<"=====
===== "<<endl;
53.                 cout<<"1. Tambah Depan"<<endl;
54.                 cout<<"2. Tambah Belakang"<<endl;
55.                 cout<<"3. Tampilkan Data"<<endl;
56.                 cout<<"4. Hapus Depan"<<endl;
57.                 cout<<"5. Hapus Belakang"<<endl;
58.                 cout<<"6. Reset"<<endl;
59.                 cout<<"7. Kembali ke Menu"<<endl;
60.                 cout<<"Pilihan : ";

```

```

61.         cin>>pilihan;
62.         pil=atoi(pilihan);
63.
64.         switch(pil) {
65.             case 1:
66.                 tambahDepanH();
67.                 break;
68.             case 2:
69.                 tambahBelakangH();
70.                 break;
71.             case 3:
72.                 tampilkanH();
73.                 break;
74.             case 4:
75.                 hapusDepanH();
76.                 break;
77.             case 5:
78.                 hapusBelakangH();
79.                 break;
80.             case 6:
81.                 clearH();
82.                 break;
83.             default:
84.                 system("cls");
85.                 goto menu;
86.         }
87.
88.         cout<<"\npress any key to continue"<<endl;
89.         getch();
90.         system("cls");
91.
92.     } while (pil<7);
93. } else if(menu==2){
94.     do {
95.         cout<<"Double Linked List Non Circular
(DLLNC) (Head dan Tail)"<<endl;
96.         cout<<"=====
===== "<<endl;
97.         cout<<"1. Tambah Depan"<<endl;
98.         cout<<"2. Tambah Belakang"<<endl;

```



```

99.         cout<<"3. Tampilkan Data"<<endl;
100.        cout<<"4. Hapus Depan"<<endl;
101.        cout<<"5. Hapus Belakang"<<endl;
102.        cout<<"6. Reset"<<endl;
103.        cout<<"7. Kembali ke Menu"<<endl;
104.        cout<<"Pilihan : ";
105.        cin>>pilihan;
106.        pil=atoi(pilihan);
107.
108.        switch(pil) {
109.            case 1:
110.                tambahDepanHT();
111.                break;
112.            case 2:
113.                tambahBelakangHT();
114.                break;
115.            case 3:
116.                tampilkanHT();
117.                break;
118.            case 4:
119.                hapusDepanHT();
120.                break;
121.            case 5:
122.                hapusBelakangHT();
123.                break;
124.            case 6:
125.                clearHT();
126.                break;
127.            default:
128.                system("cls");
129.                goto menu;
130.        }
131.
132.        cout<<"\npress any key to continue"<<endl;
133.        getch();
134.        system("cls");
135.
136.        } while (pil<7);
137.    } else {
138.        cout<<"\nTERIMA KASIH"<<endl;

```

```

139.             cout<<"    Muhammad    Rafly    Ash-Shadiq
(2510817310007) ."<<endl;
140.     }
141. }
142.
143. void initH(){
144.     head = NULL;
145. }
146.
147. void initHT(){
148.     head = NULL;
149.     tail = NULL;
150. }
151.
152. int isEmptyH(){
153.     if(head == NULL) return 1;
154.     else return 0;
155. }
156.
157. int isEmptyHT(){
158.     if(tail == NULL) return 1;
159.     else return 0;
160. }
161.
162. void tambahDepanH() {
163.     cout<<"Masukkan data : ";
164.     cin>>dataBaru;
165.     TNode *baru;
166.     baru = new TNode;
167.     baru->data = dataBaru;
168.     baru->next = NULL;
169.     baru->prev = NULL;
170.     if(isEmptyH() == 1) {
171.         head = baru;
172.     } else {
173.         baru->next = head;
174.         head->prev = baru;
175.         head = baru;
176.     }

```

```

177.         cout << "Data \""<<dataBaru<<"\" berhasil
dimasukkan di bagian depan.";
178.     }
179.
180. void tambahDepanHT() {
181.     cout<<"Masukkan data : ";
182.     cin>>dataBaru;
183.     TNode *baru;
184.     baru = new TNode;
185.     baru->data = dataBaru;
186.     baru->next = NULL;
187.     baru->prev = NULL;
188.     if(isEmptyHT() == 1) {
189.         head = baru;
190.         tail = baru;
191.     } else {
192.         baru->next = head;
193.         head->prev = baru;
194.         head = baru;
195.     }
196.     cout << "Data \""<<dataBaru<<"\" berhasil
dimasukkan di bagian depan.";
197. }
198.
199. void tambahBelakangH() {
200.     cout<<"Masukkan data : ";
201.     cin>>dataBaru;
202.     TNode *baru, *bantu;
203.     baru = new TNode;
204.     baru->data = dataBaru;
205.     baru->next = NULL;
206.     baru->prev = NULL;
207.     if(isEmptyH() == 1) {
208.         head = baru;
209.     } else {
210.         bantu = head;
211.         while(bantu->next != NULL){
212.             bantu = bantu->next;
213.         }
214.         bantu->next = baru;

```

```

215.         baru->prev = bantu;
216.     }
217.     cout << "Data \""<<dataBaru<<"\" berhasil
dimasukkan di bagian belakang.";
218. }
219.
220. void tambahBelakangHT() {
221.     cout<<"Masukkan data : ";
222.     cin>>dataBaru;
223.     TNode *baru;
224.     baru = new TNode;
225.     baru->data = dataBaru;
226.     baru->next = NULL;
227.     baru->prev = NULL;
228.     if(isEmptyHT() == 1) {
229.         head = baru;
230.         tail = baru;
231.     } else {
232.         tail->next = baru;
233.         baru->prev = tail;
234.         tail = baru;
235.     }
236.     cout << "Data \""<<dataBaru<<"\" berhasil
dimasukkan di bagian belakang.";
237. }
238.
239. void tampilkanH() {
240.     TNode *bantu;
241.     bantu = head;
242.     if(isEmptyH() == 0) {
243.         while(bantu != NULL) {
244.             cout<<head->data<<' ';
245.             bantu = bantu->next;
246.         }
247.         cout<<endl;
248.     } else cout<<"Tidak terdapat data pada Linked
List";
249. }
250.
251. void tampilkanHT() {

```

```

252.     TNode *bantu;
253.     bantu = head;
254.     if(isEmptyHT() == 0) {
255.         while(bantu != tail->next) {
256.             cout<<head->data<<' ';
257.             bantu = bantu->next;
258.         }
259.         cout<<endl;
260.     } else cout<<"Tidak terdapat data pada Linked
List";
261. }
262.
263. void hapusDepanH() {
264.     TNode *hapus;
265.     string data;
266.     if(isEmptyH() == 0) {
267.         hapus = head;
268.         data = hapus->data;
269.         if(head->next != NULL) {
270.             head = head->next;
271.             head->prev = NULL;
272.         } else {
273.             initH();
274.         }
275.         delete hapus;
276.         cout<<"Data \""<<data<<"\" yang berada di depan
telah berhasil dihapus.";
277.     } else cout<<"Tidak terdapat data pada Linked
List";
278. }
279.
280. void hapusDepanHT() {
281.     TNode *hapus;
282.     string data;
283.     if(isEmptyHT() == 0) {
284.         hapus = head;
285.         data = hapus->data;
286.         if(head->next != NULL) {
287.             head = head->next;
288.             head->prev = NULL;

```

```

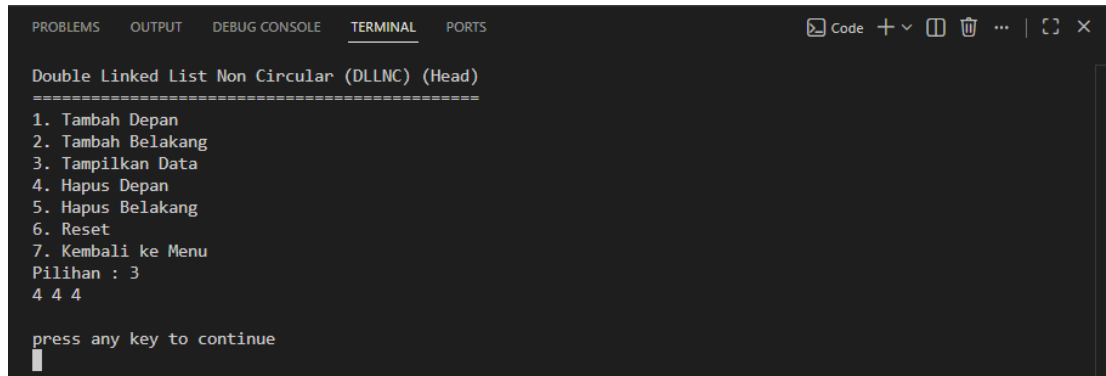
289.         } else {
290.             initHT();
291.         }
292.         delete hapus;
293.         cout<<"Data \""<<data<<"\" yang berada di depan
telah berhasil dihapus.";
294.     } else cout<<"Tidak terdapat data pada Linked
List";
295. }
296.
297. void hapusBelakangH() {
298.     TNode *hapus;
299.     string data;
300.     if(isEmptyH() == 0) {
301.         hapus = head;
302.         while(hapus->next != NULL){
303.             hapus = hapus->next;
304.         }
305.         data = hapus->data;
306.         if(head->next != NULL) {
307.             hapus->prev->next = NULL;
308.         } else {
309.             initH();
310.         }
311.         delete hapus;
312.         cout<<"Data \""<<data<<"\" yang berada di
belakang telah berhasil dihapus.";
313.     } else cout<<"Tidak terdapat data pada Linked
List";
314. }
315.
316. void hapusBelakangHT() {
317.     TNode *hapus;
318.     string data;
319.     if(isEmptyHT() == 0) {
320.         hapus = tail;
321.         data = hapus->data;
322.         if(head->next != NULL) {
323.             tail = tail->prev;
324.             tail->next = NULL;

```

```

325.         } else {
326.             initHT();
327.         }
328.         delete hapus;
329.         cout<<"Data \""<<data<<"\" yang berada di
belakang telah berhasil dihapus.";
330.     } else cout<<"Tidak terdapat data pada Linked
List";
331. }
332.
333. void clearH() {
334.     TNode *bantu, *hapus;
335.     bantu = head;
336.     while(bantu != NULL) {
337.         hapus = bantu;
338.         bantu = bantu->next;
339.         delete hapus;
340.     }
341.     initH();
342.     cout<<"Seluruh data pada Linked List telah
dibersihkan.";
343. }
344.
345. void clearHT() {
346.     TNode *bantu, *hapus;
347.     bantu = head;
348.     while(bantu != NULL) {
349.         hapus = bantu;
350.         bantu = bantu->next;
351.         delete hapus;
352.     }
353.     initHT();
354.     cout<<"Seluruh data pada Linked List telah
dibersihkan.";
355. }

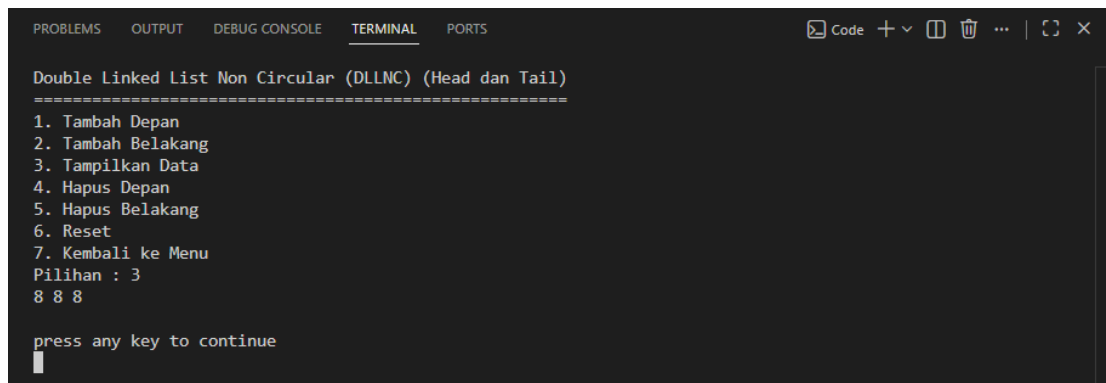
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Double Linked List Non Circular (DLLNC) (Head)
=====
1. Tambah Depan
2. Tambah Belakang
3. Tampilkan Data
4. Hapus Depan
5. Hapus Belakang
6. Reset
7. Kembali ke Menu
Pilihan : 3
4 4 4

press any key to continue
```

Gambar 2 Screenshot Output Soal 4



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Double Linked List Non Circular (DLLNC) (Head dan Tail)
=====
1. Tambah Depan
2. Tambah Belakang
3. Tampilkan Data
4. Hapus Depan
5. Hapus Belakang
6. Reset
7. Kembali ke Menu
Pilihan : 3
8 8 8

press any key to continue
```

Gambar 3 Screenshot Output Soal 4

Hal tersebut terjadi karena pada baris output yang telah diubah, yaitu dari `cout<<bantu->data` menjadi `cout<<head->data`, program tidak lagi menampilkan data dari node yang sedang ditelusuri (`bantu`), melainkan selalu menampilkan data yang ada pada node pertama (`head`). Padahal, pada proses traversal linked list, variabel `bantu` berfungsi untuk berpindah dari satu node ke node berikutnya menggunakan pointer `next`, sehingga seharusnya setiap node yang dilewati akan ditampilkan datanya secara berurutan.

Namun karena yang dicetak adalah `head->data`, maka walaupun `bantu` tetap bergerak ke node berikutnya (loop tetap berjalan sesuai jumlah node), nilai yang

ditampilkan tidak ikut berubah, karena head selalu menunjuk ke node pertama. Akibatnya, output yang muncul adalah pengulangan data pertama sebanyak jumlah node dalam linked list.

Sebagai contoh pada percobaan pertama (DLLNC dengan Head), data yang dimasukkan adalah 3, 4, dan 5. Secara logika linked list, urutan datanya menjadi $4 \rightarrow 3 \rightarrow 5$ karena tambah depan dan belakang. Namun yang ditampilkan adalah 4 4 4, karena, head menunjuk ke node dengan data 4, perulangan berjalan sebanyak 3 node, tetapi setiap iterasi selalu mencetak head->data (4)

Hal yang sama juga terjadi pada percobaan kedua (DLLNC dengan Head dan Tail), di mana data yang dimasukkan adalah 7, 8, dan 9, tetapi output yang muncul adalah 8 8 8, karena, head menunjuk ke node dengan data 8, perulangan tetap berjalan mengikuti jumlah node, namun yang ditampilkan selalu data dari head

Dengan demikian, dapat disimpulkan bahwa program hanya menampilkan data dari node pertama (head) secara berulang, bukan seluruh isi linked list. Kesalahan ini terjadi karena penggunaan pointer yang tidak sesuai dalam proses output, di mana seharusnya menggunakan bantu->data agar data yang ditampilkan mengikuti proses traversal node.

LINK GITHUB

<https://github.com/Arthur-369/PRAKTIKUM-Algoritma-dan-Struktur-Data>